STAT 432 Week 11 Homework: K-means and Hierarchical Clustering

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from skimage import data
from skimage.transform import resize

pd.set_option("display.precision", 3)
pd.set_option("display.width", 140)
```

Question 1: Implementing K-means

Data and random initial assignment

The data-generating seed is **432**. The initial assignment draws each label uniformly from {0, 1} and redraws until both clusters are nonempty.

In [2]:
```
SEED = 432
rng = np.random.default_rng(SEED)
X = rng.standard_normal((10, 2))

def random_assignment(n, K, rng):
    """Uniform random labels in {0,...,K-1}, redrawn until every cluster is nonempty."""
    while True:
        C = rng.integers(0, K, size=n)
        if len(np.unique(C)) == K:
            return C

C0 = random_assignment(10, 2, np.random.default_rng(SEED))
pd.DataFrame(X, columns=["x1", "x2"]).assign(initial_cluster=C0)
```

Out[2]:

	x1	x2	initial_cluster
0	0.708	-1.298	1
1	-1.352	-0.064	1
2	0.086	-0.583	0
3	-1.115	0.238	1
4	-0.486	1.806	1
5	-0.534	-1.988	0
6	2.127	-0.951	1
7	-0.812	0.358	1
8	0.278	0.778	1
9	0.725	-1.760	0

1.1 One iteration

Step 1 computes each cluster mean from the current assignment. Step 2 reassigns each observation to its nearest mean using Euclidean distance.

In [3]:

```
def cluster_means(X, C, K, prev_means=None):
    """Mean of each cluster. An empty cluster keeps its previous mean."""
    means = np.zeros((K, X.shape[1])) if prev_means is None else
prev_means.copy()
    for k in range(K):
        if np.any(C == k):
            means[k] = X[C == k].mean(axis=0)
    return means

def assign_to_nearest(X, means):
    """Index of the closest mean (squared Euclidean distance) for every row
of X."""
    d2 = ((X[:, None, :] - means[None, :, :]) ** 2).sum(axis=2)
    return d2.argmin(axis=1)

m1 = cluster_means(X, C0, 2)
C1 = assign_to_nearest(X, m1)

print("Cluster means from the initial assignment:")
print(pd.DataFrame(m1, columns=["x1", "x2"], index=["cluster 0", "cluster
1"]))
print("\nInitial assignment:", C0)
print("Updated assignment:", C1)
print("Observations that changed cluster:", np.where(C0 != C1)[0].tolist())
```

```
Cluster means from the initial assignment:
              x1     x2
cluster 0  0.092 -1.443
cluster 1 -0.093  0.124

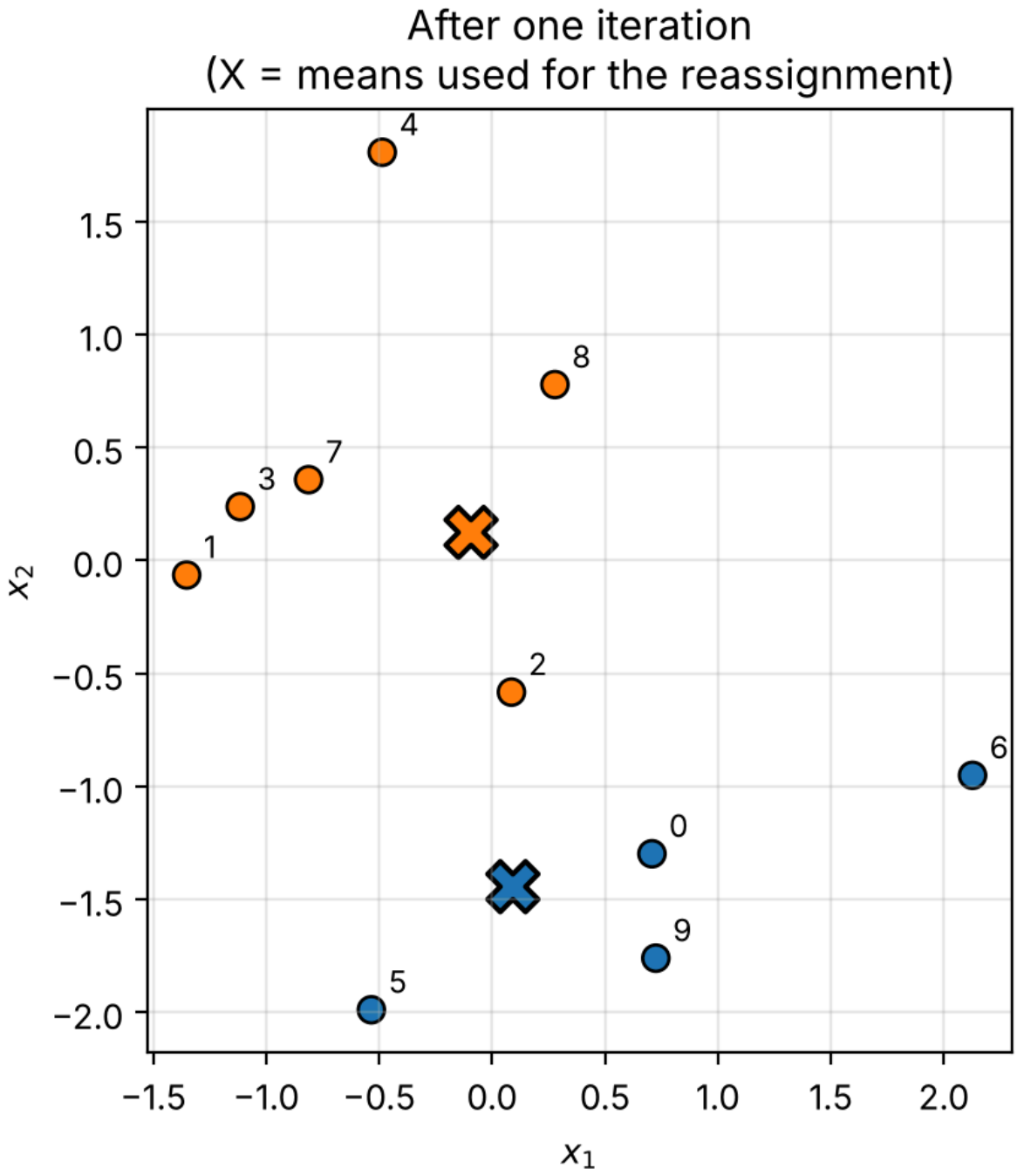
Initial assignment: [1 1 0 1 1 0 1 1 1 0]
Updated assignment: [0 1 1 1 1 0 0 1 1 0]
Observations that changed cluster: [0, 2, 6]
```

In [4]:

```
colors = np.array(["tab:blue", "tab:orange", "tab:green", "tab:red",
"tab:purple"])

def plot_clusters(X, C, means, title, ax):
    ax.scatter(X[:, 0], X[:, 1], c=colors[C], s=60, edgecolor="k")
    ax.scatter(means[:, 0], means[:, 1], c=colors[:len(means)], marker="X",
s=220,
               edgecolor="k", linewidth=1.5)
    for i, (a, b) in enumerate(X):
        ax.annotate(str(i), (a, b), xytext=(5, 5), textcoords="offset
points", fontsize=9)
    ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$"); ax.set_title(title)
    ax.set_aspect("equal"); ax.grid(alpha=0.3)

fig, ax = plt.subplots(figsize=(5.5, 5))
plot_clusters(X, C1, m1, "After one iteration\n(X = means used for the
reassignment)", ax)
plt.show()
```



1.2 The K-means function

The function alternates the two steps until the assignments stop changing or `max_iter` is reached. An empty cluster keeps its previous mean. The means are recomputed from the final assignment before they are returned.

In [5]:
```
def my_kmeans(X, K, init_assign, max_iter=100):
    """
    Lloyd's K-means algorithm.

    X           : (n, p) data matrix
    K           : number of clusters
```

```
    init_assign : length-n integer labels in {0,...,K-1}; each cluster must
be nonempty
    max_iter    : maximum number of (mean update, reassignment) iterations

    Returns a dict with the final assignments, cluster means, within-
cluster
    sum of squares, number of iterations, and whether the assignments
converged.
    """
    X = np.asarray(X, dtype=float)
    C = np.asarray(init_assign).copy()
    if X.ndim != 2 or len(C) != X.shape[0]:
        raise ValueError("X must be n x p and init_assign must have length
n")
    if set(np.unique(C)) != set(range(K)):
        raise ValueError("initial assignment must use every cluster
0,...,K-1")

    means = cluster_means(X, C, K)
    converged = False
    for it in range(1, max_iter + 1):
        new_C = assign_to_nearest(X, means)
        if np.array_equal(new_C, C):
            converged = True
            break
        C = new_C
        means = cluster_means(X, C, K, prev_means=means)

    means = cluster_means(X, C, K, prev_means=means)   # recompute from
final assignments
    wss = sum(((X[C == k] - means[k]) ** 2).sum() for k in range(K))
    return {"cluster": C, "means": means, "wss": wss,
            "iterations": it, "converged": converged}
```

Diagnostic check. At convergence each observation should be closest to its own cluster's mean, and the WSS should never increase from one iteration to the next. The cell below checks the first property and also confirms that the function works for p = 3 and K = 4.

In [6]:
```
chk_X = np.random.default_rng(1).standard_normal((50, 3))
chk = my_kmeans(chk_X, 4, random_assignment(50, 4,
np.random.default_rng(2)))
print("p=3, K=4 run: converged =", chk["converged"], "| iterations =",
chk["iterations"],
      "| WSS =", round(chk["wss"], 4))
print("Every point closest to its own mean:",
      np.array_equal(assign_to_nearest(chk_X, chk["means"]),
chk["cluster"]))
```

```
p=3, K=4 run: converged = True | iterations = 3 | WSS = 56.87
Every point closest to its own mean: True
```

1.3 Two different random initial assignments

Run A uses initialization seed 1. To show that K-means can stop at a local optimum, the code below also tries 200 initialization seeds on the same data. Run B uses the first seed that converges to a different solution. The seed is found by the search rather than picked by hand, and it is printed.

In [7]:
```
results = []
for s in range(1, 201):
    r = my_kmeans(X, 2, random_assignment(10, 2, np.random.default_rng(s)))
    results.append((s, round(r["wss"], 4)))
tally = pd.Series([w for _, w in
results]).value_counts().rename_axis("final WSS").rename("number of
starts")
print(tally, "\n")

best_wss = min(w for _, w in results)
seed_A = 1
seed_B = next(s for s, w in results if w > best_wss)
print("Run A init seed:", seed_A, "| Run B init seed:", seed_B)
```

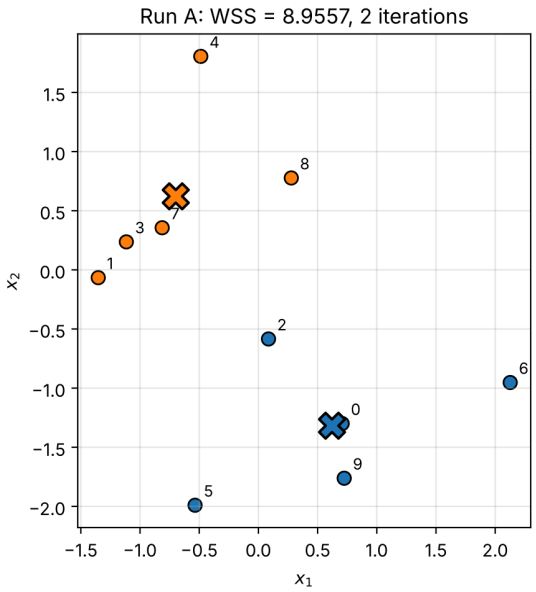
```
final WSS
8.956     191
17.341      9
Name: number of starts, dtype: int64

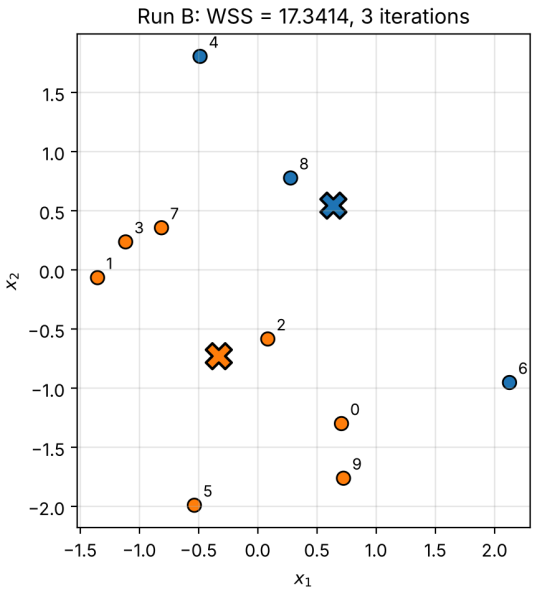
Run A init seed: 1 | Run B init seed: 33
```

In [8]:
```
runs = {}
for name, s in [("A", seed_A), ("B", seed_B)]:
    init = random_assignment(10, 2, np.random.default_rng(s))
    runs[name] = (init, my_kmeans(X, 2, init))

fig, axes = plt.subplots(1, 2, figsize=(11, 5))
for ax, (name, (init, r)) in zip(axes, runs.items()):
    plot_clusters(X, r["cluster"], r["means"],
                  f"Run {name}: WSS = {r['wss']:.4f}, {r['iterations']}
iterations", ax)
plt.tight_layout(); plt.show()

for name, (init, r) in runs.items():
    print(f"Run {name}")
    print("  initial assignment:", init)
    print("  final assignment:  ", r["cluster"])
    print("  cluster means:\n", pd.DataFrame(r["means"], columns=["x1",
"x2"],
                                             index=["cluster 0", "cluster
1"]).to_string())
    print(f"  WSS = {r['wss']:.4f}, converged = {r['converged']}\n")
```






```
Run A
  initial assignment: [0 1 1 1 0 0 1 1 0 0]
  final assignment:   [0 1 0 1 1 0 0 1 1 0]
  cluster means:
                x1     x2
cluster 0  0.622 -1.316
cluster 1 -0.697  0.623
  WSS = 8.9557, converged = True

Run B
  initial assignment: [1 0 0 1 1 1 0 0 1 1]
  final assignment:   [1 1 1 1 0 1 0 1 0 1]
  cluster means:
                x1     x2
cluster 0  0.640  0.545
cluster 1 -0.328 -0.728
  WSS = 17.3414, converged = True
```

Discussion. The two runs do **not** give the same grouping or the same objective value.

- Run A converges to WSS = 8.956. It splits the data into a lower-right group {0, 2, 5, 6, 9} and an upper-left group {1, 3, 4, 7, 8}. This is also the smallest WSS found across all 200 random starts, which reached it 191 times.
- Run B converges to WSS = 17.341, almost twice as large. The upper-right points {4, 6, 8} form one cluster and the other seven points form the second.

Run B really is a fixed point of the algorithm. Every observation is closest to its own cluster mean, and each mean is the average of its points, so neither step can change anything and the algorithm stops. K-means only guarantees that the WSS does not increase from one iteration to the next. That means it converges to a *local* minimum, and which one it reaches depends on the starting assignment. Convergence alone does not mean the objective is as small as possible. In practice we run K-means from many random starts and keep the solution with the lowest WSS (here, Run A).

The cluster *labels* are arbitrary. Two runs with the same partition but swapped labels 0 and 1 would be the same clustering. So we compare runs by their partitions and WSS values, not by their label vectors.

Question 2: Image color compression

The image is the `coffee` photo that ships with scikit-image, a cup of coffee on a saucer with a colorful background. It is resized to 133 × 200 pixels, which keeps the original aspect ratio and stays within the 200 × 200 limit. RGB values are on the [0, 1] scale.

In [9]:
```
img_full = data.coffee()
img = resize(img_full, (133, 200), anti_aliasing=True)   # floats in [0, 1]
print("Original file dimensions:", img_full.shape)
print("Resized image dimensions (height, width, channels):", img.shape)

plt.figure(figsize=(6, 4))
plt.imshow(img); plt.axis("off"); plt.title("Original (resized to 133 × 200)")
plt.show()

H, W, _ = img.shape
pixels = img.reshape(-1, 3)
print("Pixel data matrix dimensions:", pixels.shape, "(one row per pixel; columns R, G, B)")
```

```
Original file dimensions: (400, 600, 3)
Resized image dimensions (height, width, channels): (133, 200, 3)
```




```
Pixel data matrix dimensions: (26600, 3) (one row per pixel; columns R, G,
B)
```

2.1 to 2.2 K-means for K = 2, 3, 5, 10

Each fit starts from a random nonempty assignment with seed 432. Every reconstruction is drawn with `imshow` on the same fixed [0, 1] RGB scale.

In [10]:
```
Ks = [2, 3, 5, 10]
fits = {}
for K in Ks:
    init = random_assignment(len(pixels), K, np.random.default_rng(SEED))
    fits[K] = my_kmeans(pixels, K, init, max_iter=300)
    r = fits[K]
    print(f"K = {K}: WSS = {r['wss']:.2f}, iterations = {r['iterations']},
converged = {r['converged']}")
    print(pd.DataFrame(r["means"], columns=["R", "G", "B"])
            .assign(n_pixels=np.bincount(r["cluster"],
minlength=K)).to_string(), "\n")
```

```
K = 2: WSS = 1793.71, iterations = 21, converged = True
       R      G      B  n_pixels
0  0.457  0.151  0.065     13647
1  0.796  0.532  0.346     12953
```

```
K = 3: WSS = 858.06, iterations = 20, converged = True
       R      G      B  n_pixels
0  0.226  0.057  0.027      5976
1  0.861  0.662  0.490      6082
2  0.684  0.315  0.153     14542

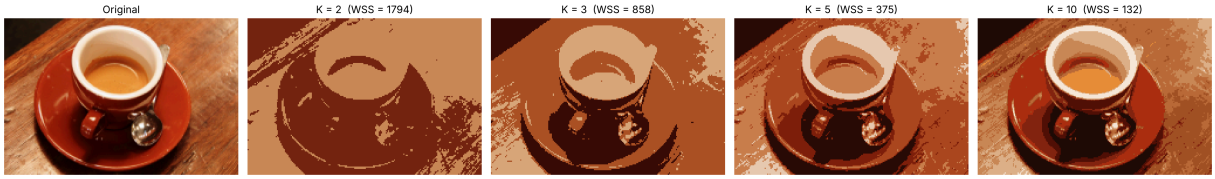
K = 5: WSS = 375.50, iterations = 31, converged = True
       R      G      B  n_pixels
0  0.493  0.129  0.053      2950
1  0.671  0.280  0.125      8771
2  0.917  0.801  0.689      2463
3  0.166  0.043  0.022      4570
4  0.788  0.503  0.295      7846

K = 10: WSS = 131.89, iterations = 62, converged = True
       R      G      B  n_pixels
0  0.327  0.072  0.032      1651
1  0.743  0.421  0.222      4802
2  0.137  0.040  0.020      3661
3  0.962  0.908  0.850      1042
4  0.875  0.702  0.545      1898
5  0.905  0.567  0.218       800
6  0.641  0.315  0.150      4162
7  0.520  0.147  0.060      2021
8  0.789  0.539  0.353      3129
9  0.686  0.186  0.067      3434
```

In [11]:

```
fig, axes = plt.subplots(1, 5, figsize=(20, 3.6))
axes[0].imshow(img, vmin=0, vmax=1); axes[0].set_title("Original");
axes[0].axis("off")
for ax, K in zip(axes[1:], Ks):
    r = fits[K]
    recon = r["means"][r["cluster"]].reshape(H, W, 3)
    ax.imshow(np.clip(recon, 0, 1), vmin=0, vmax=1)
    ax.set_title(f"K = {K}  (WSS = {r['wss']:.0f})"); ax.axis("off")
plt.tight_layout(); plt.show()

wss_tab = pd.DataFrame({"K": Ks, "WSS": [fits[K]["wss"] for K in Ks]})
wss_tab["% of K=2 WSS"] = 100 * wss_tab["WSS"] / wss_tab["WSS"].iloc[0]
tss = ((pixels - pixels.mean(0)) ** 2).sum()
wss_tab["% of total SS explained"] = 100 * (1 - wss_tab["WSS"] / tss)
print(f"Total sum of squares (K = 1): {tss:.2f}")
wss_tab
```



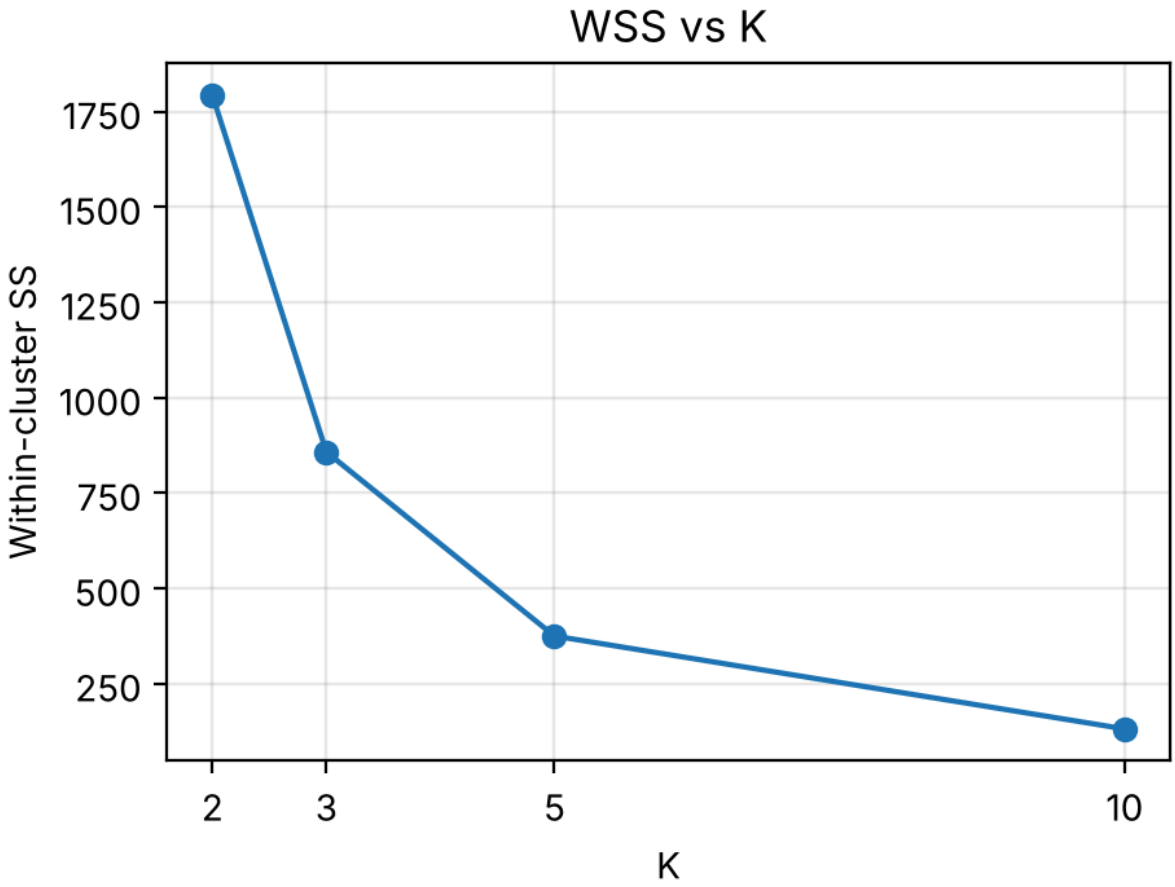

```
Total sum of squares (K = 1): 4040.91
```

Out[11]:

	K	WSS	% of K=2 WSS	% of total SS explained
0	2	1793.714	100.000	55.611
1	3	858.062	47.837	78.766
2	5	375.500	20.934	90.708
3	10	131.885	7.353	96.736

In [12]:

```
plt.figure(figsize=(5, 3.5))
plt.plot(wss_tab["K"], wss_tab["WSS"], "o-")
plt.xticks(Ks); plt.xlabel("K"); plt.ylabel("Within-cluster SS");
plt.title("WSS vs K")
plt.grid(alpha=0.3); plt.show()
```



2.3 Choice of K

I would choose K = 10. The WSS falls from 1794 (K = 2) to 858 (K = 3), 376 (K = 5), and 132 (K = 10). That is 56%, 79%, 91%, and 97% of the total sum of squares. The returns shrink as K grows, but the images show that the last step still matters.

- K = 2 and K = 3 reduce the photo to a dark/light silhouette. The cup and saucer are recognizable but the colors are wrong.

- K = 5 recovers the white cup rim, the red saucer, and the wood grain. However, the espresso crema gets the same tan color as the table, and the crema is the focal point of the image.
- K = 10 brings back the orange crema, the shading on the saucer, and the highlights on the spoon. It looks close to the original while still using only 10 colors instead of tens of thousands.

Going past 10 would keep lowering the WSS (it reaches 0 when K equals the number of distinct pixels), but the visual gain would be small. A smaller WSS alone is never enough to justify a larger K. Here K = 10 is the point where the reconstruction stops having visible errors in the main subject. If storage were very tight, K = 5 would be the fallback because it already explains 91% of the variation.

Question 3: Hierarchical clustering of cars

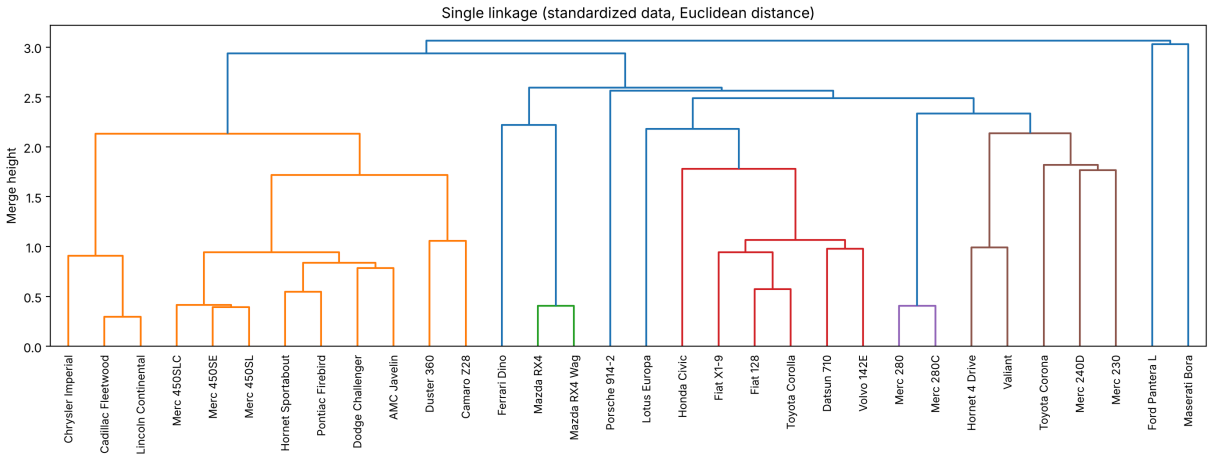
3.1 Standardize, then use single, complete, and average linkage

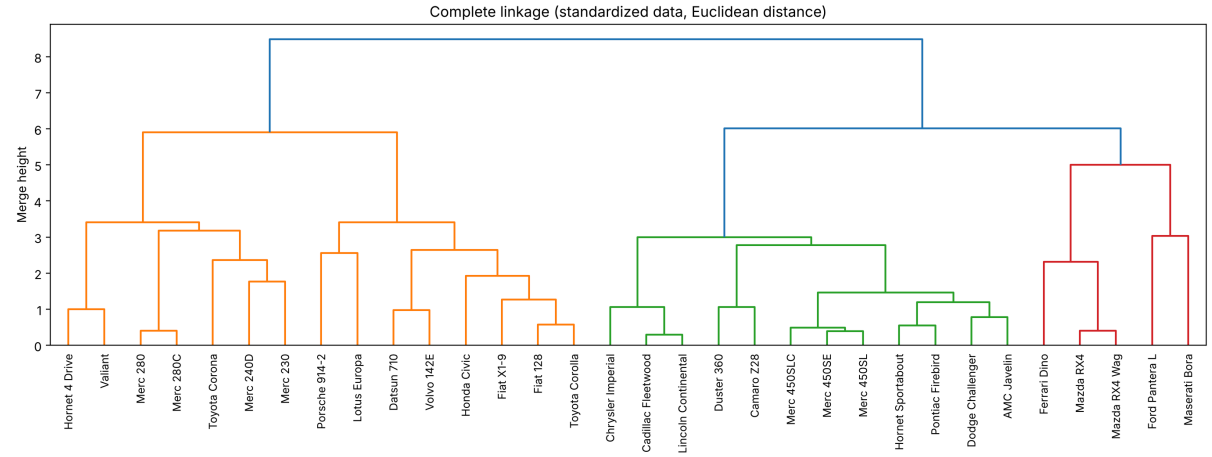
In [13]:
```
cars = pd.read_csv("data/mtcars.csv")
names = cars["car"].to_numpy()
num = cars.drop(columns="car")
Z = (num - num.mean()) / num.std(ddof=1)        # mean 0, SD 1 per variable
print(Z.agg(["mean", "std"]).round(3))
```

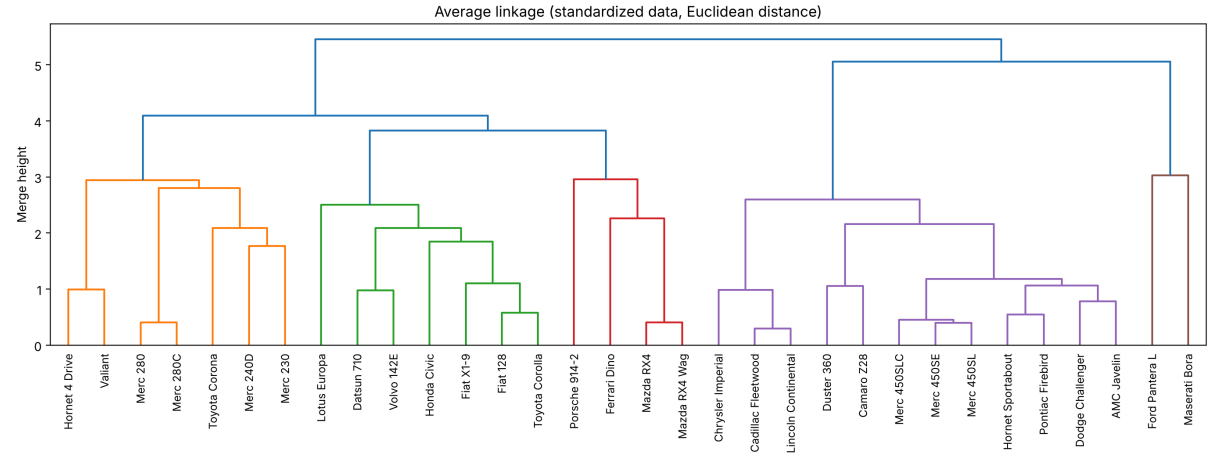
```
      mpg  cyl  disp   hp  drat   wt  qsec   vs   am  gear  carb
mean -0.0 -0.0   0.0  0.0  -0.0 -0.0  -0.0  0.0  0.0  -0.0   0.0
std   1.0  1.0   1.0  1.0   1.0  1.0   1.0  1.0  1.0   1.0   1.0
```

In [14]:
```
links = {m: linkage(Z.to_numpy(), method=m, metric="euclidean")
         for m in ["single", "complete", "average"]}

fig, axes = plt.subplots(3, 1, figsize=(14, 16))
for ax, (m, L) in zip(axes, links.items()):
    dendrogram(L, labels=names, leaf_rotation=90, leaf_font_size=9, ax=ax,
               color_threshold=None)
    ax.set_title(f"{m.capitalize()} linkage (standardized data, Euclidean
distance)")
    ax.set_ylabel("Merge height")
plt.tight_layout(); plt.show()
```







In [15]:
```
# Cluster sizes when each tree is cut into 2 to 4 groups, to help compare the linkages
rows = []
for m, L in links.items():
    for k in [2, 3, 4]:
        sizes = sorted(np.bincount(fcluster(L, k, criterion="maxclust"))[1:], reverse=True)
        rows.append({"linkage": m, "k": k, "cluster sizes": sizes})
pd.DataFrame(rows).pivot(index="linkage", columns="k", values="cluster sizes")
```

Out[15]:

k	2	3	4
linkage			
average	[18, 14]	[18, 12, 2]	[12, 11, 7, 2]
complete	[17, 15]	[15, 12, 5]	[12, 8, 7, 5]
single	[30, 2]	[30, 1, 1]	[18, 12, 1, 1]

In [16]:
```
# Which cars are split off as singletons by single linkage at k = 4?
lab = fcluster(links["single"], 4, criterion="maxclust")
for c in np.unique(lab):
    if (lab == c).sum() <= 3:
        print(f"single-linkage small cluster {c}:", names[lab ==
c].tolist())
```

```
single-linkage small cluster 3: ['Ford Pantera L']
single-linkage small cluster 4: ['Maserati Bora']
```

Comparison of the three dendrograms.

- **Single linkage** shows heavy *chaining*. Clusters grow by adding one car at a time at slowly increasing heights, and most merges happen in a narrow band between about 1.8 and 3. The Ford Pantera L and the Maserati Bora join only at the very top, so cutting into 2 to 3 clusters gives one cluster of 30 cars plus singletons ([30, 2], then [30, 1, 1]). Single linkage merges groups based on their *closest* pair, which makes it sensitive to unusual observations and poor at finding compact groups.
- **Complete linkage** gives compact, well-separated groups with clear jumps in merge height. The top split (height ≈ 8.5) separates light, economical cars from heavy or high-performance ones. Next comes a split at about 6 between the American V8 sedans and muscle cars and a group of five sporty European/Japanese cars (Ferrari Dino, Mazda RX4, RX4 Wag, Ford Pantera L, Maserati Bora). Then the economical cars split at about 5.9.
- **Average linkage** falls between the two. Its main V8 group and economy groups are almost the same as complete linkage. The differences are that Pantera and Bora form their own pair, which stays separate until a late merge, and that Ferrari Dino and the Mazdas join the Porsche/Lotus group. This gives sizes [18, 12, 2] at k = 3.

All three methods agree on the tightest pairs (Merc 280/280C, Mazda RX4/RX4 Wag, the three Merc 450s, Cadillac/Lincoln). They differ mostly on where the few extreme cars go.

3.2 Chosen clustering: complete linkage, 3 clusters

In [17]:
```
cl = fcluster(links["complete"], 3, criterion="maxclust")
memb = pd.DataFrame({"car": names, "cluster": cl}).sort_values(["cluster",
```

```
"car"])
for c, g in memb.groupby("cluster"):
    print(f"Cluster {c} ({len(g)} cars): " + ", ".join(g["car"]))
```

```
Cluster 1 (15 cars): Datsun 710, Fiat 128, Fiat X1-9, Honda Civic, Hornet 4
Drive, Lotus Europa, Merc 230, Merc 240D, Merc 280, Merc 280C, Porsche 914-
2, Toyota Corolla, Toyota Corona, Valiant, Volvo 142E
Cluster 2 (12 cars): AMC Javelin, Cadillac Fleetwood, Camaro Z28, Chrysler
Imperial, Dodge Challenger, Duster 360, Hornet Sportabout, Lincoln
Continental, Merc 450SE, Merc 450SL, Merc 450SLC, Pontiac Firebird
Cluster 3 (5 cars): Ferrari Dino, Ford Pantera L, Maserati Bora, Mazda RX4,
Mazda RX4 Wag
```

In [18]:
```
means_tab = num.assign(cluster=cl).groupby("cluster").mean()
means_tab.insert(0, "n",
pd.Series(cl).value_counts().sort_index().to_numpy())
means_tab.round(2)
```

Out[18]:

cluster	n	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
1	15	24.65	4.53	131.65	91.33	3.90	2.58	19.16	0.93	0.53	3.93	1.80
2	12	15.05	8.00	357.62	194.17	3.12	4.10	17.14	0.00	0.00	3.00	3.08
3	5	18.50	6.80	223.40	198.80	3.84	3.00	15.62	0.00	1.00	4.60	5.20

In [19]:
```
# Standardized cluster means: which variables separate the groups most?
Z.assign(cluster=cl).groupby("cluster").mean().round(2)
```

Out[19]:

cluster	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
1	0.76	-0.93	-0.80	-0.81	0.56	-0.65	0.73	0.98	0.25	0.33	-0.63
2	-0.84	1.01	1.02	0.69	-0.89	0.91	-0.40	-0.87	-0.81	-0.93	0.17
3	-0.26	0.34	-0.06	0.76	0.45	-0.22	-1.25	-0.87	1.19	1.24	1.48

In [20]:
```
# Why standardizing matters: spread of each raw variable, and what an
unscaled
# complete-linkage clustering keys on.
print(num.std().sort_values(ascending=False).round(2), "\n")
raw_cl = fcluster(linkage(num.to_numpy(), method="complete"), 3,
criterion="maxclust")
print("Unscaled complete linkage, 3 clusters, mean disp and hp by
cluster:")
print(num.assign(cluster=raw_cl).groupby("cluster")[["disp", "hp", "mpg",
"wt"]].mean().round(1))
print("\nCross-tab, standardized vs unscaled cluster labels:")
print(pd.crosstab(pd.Series(cl, name="standardized"), pd.Series(raw_cl,
name="unscaled")))
```

```
disp    123.94
hp       68.56
mpg       6.03
qsec      1.79
cyl       1.79
carb      1.62
wt        0.98
gear      0.74
drat      0.53
vs        0.50
am        0.50
dtype: float64

Unscaled complete linkage, 3 clusters, mean disp and hp by cluster:
          disp     hp   mpg   wt
cluster
1        388.2  232.1  14.6  4.2
2        276.1  150.7  17.0  3.6
3        122.3   96.9  24.5  2.5

Cross-tab, standardized vs unscaled cluster labels:
unscaled      1  2   3
standardized
1             0  2  13
2             7  5   0
3             2  0   3
```

Choice and interpretation. I chose **complete linkage with 3 clusters**. In that dendrogram, cutting at a height of about 5.5 crosses three branches. Below the cut, the economy cars and the V8 cars have no merges above about 3.4, and the last merge inside the sporty group is at about 5.0. The next merges above the cut are at about 5.9 and 6.0, and the top merge is at 8.5. Stopping at 3 avoids the 8.5 merge that would combine very different cars, and a fourth cluster would only split the five sporty cars into two very small groups. The three groups also have reasonable sizes (15, 12, 5) and clear interpretations. Single linkage would give a 30-car blob plus singletons, which is not useful.

Cluster	Cars	Main characteristics (original scale)
1. Economical cars (15)	Small Japanese, European, and compact US cars (Civic, Corolla, Fiat 128, Mercs 230/240D/280, Volvo, Hornet 4 Drive, Valiant, ...)	Highest mpg (24.7), mostly 4 cylinders, small engines (disp 132, 91 hp), light (2.6k lb), mostly straight engines (vs = 0.93), slowest quarter mile (19.2 s)
2. Large V8 cruisers (12)	Cadillac, Lincoln, Chrysler Imperial, Merc 450s, Camaro, Duster, Javelin, Challenger, Firebird, Sportabout	All 8 cylinders, all automatic, 3 gears, largest engines (disp 358, 194 hp), heaviest (4.1k lb), lowest mpg (15.1), low rear-axle ratio
3. Sporty manual cars (5)	Ferrari Dino, Maserati Bora, Ford Pantera L, Mazda RX4, RX4 Wag	All manual, 4.6 gears and 5.2 carburetors on average, hp (199) as high as the V8s but much lighter (3.0k lb) and smaller engines (223), fastest quarter mile (15.6 s), middle mpg (18.5)

The standardized means table confirms this. Cluster 1 is low on cyl/disp/hp/wt and high on mpg/vs. Cluster 2 is the opposite. Cluster 3 stands out on am, gear, carb (all above +1.2 SD), and qsec (−1.25 SD).

Why standardizing matters. Euclidean distance adds up squared differences across variables, so a variable with a large numeric spread dominates. In the raw data, disp has SD 124 and hp has SD 69, while wt has SD 0.98 and vs and am have SD 0.5. Without scaling, a 1-unit change in displacement counts as much as switching from automatic to manual, which makes no sense when comparing cars. As a result, the unscaled clustering above sorts cars almost entirely by engine size and horsepower. The cross-tab shows it splits the large-V8 cluster and mixes the sporty manual cars in with the others. Standardizing puts every variable in SD units, so each variable has comparable influence, and transmission, gears, and carburetors can contribute. That is what makes the sporty-manual group visible. Standardizing is still a choice, though. It gives equal weight to all 11 variables, including correlated ones (cyl, disp, hp, wt all measure size), so the "size" dimension still has more total weight than the others.